PRAKTIKUM BASIS DATA

MODUL 09

Scripting Procedure Function

Oleh:

Danu Warisman 103122400041

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

Nomor 1

Pada langkah pertama ini, saya menggunakan perintah create table employees as select dari hr.employees. Logika dari perintah ini adalah untuk membuat sebuah tabel baru di dalam skema saya sendiri dengan nama employees. Tabel baru ini tidak dibuat dari nol, melainkan langsung menyalin seluruh struktur kolom sekaligus isi data dari tabel employees yang sudah ada di skema bawaan HR. Pendekatan ini sangat aman karena saya bisa melakukan berbagai uji coba manipulasi data di tabel baru ini tanpa khawatir merusak data asli milik HR

```
CREATE TABLE employees (
    employee_id NUMBER(6) PRIMARY KEY,
    first_name VARCHAR2(20),
    last_name VARCHAR2(25) NOT NULL,
    email VARCHAR2(25) NOT NULL,
    phone_number VARCHAR2(20),
    hire_date DATE NOT NULL,
    job_id VARCHAR2(10) NOT NULL,
    salary NUMBER(8,2),
    commission_pct NUMBER(2,2),
    manager_id NUMBER(6),
    department_id NUMBER(4)
);

INSERT INTO employees VALUES (100, 'Steven', 'King', 'SKING', '515.123.4567', TO_DATE('17-JUN-03', 'dd-MON-yy'), 'AD_PRES', 24000, NULL, NULL, 90);
INSERT INTO employees VALUES (101, 'Neena', 'Kochhar', 'NKOCHHAR', '515.123.4568', TO_DATE('21-SEP-05', 'dd-MON-yy'), 'AD_VP', 17000, NULL, 100, 90);
INSERT INTO employees VALUES (102, 'Lex', 'De Haan', 'LDEHAAN', '515.123.4569', TO_DATE('13-JAN-01', 'dd-MON-yy'), 'AD_VP', 17000, NULL, 100, 90);
INSERT INTO employees VALUES (103, 'Alexander', 'Hunold', 'AHUNOLD', '590.423.4567', TO_DATE('03-JAN-06', 'dd-MON-yy'), 'IT_PROG', 9000, NULL, 102, 60);
INSERT INTO employees VALUES (104, 'Bruce', 'Ernst', 'BERNST', '590.423.4568', TO_DATE('21-MAY-07', 'dd-MON-yy'), 'IT_PROG', 6000, NULL, 103, 60);

COMMIT;
```

Script Output | Task completed in 0.111 seconds

```
Table EMPLOYEES created.
```

Nomor 2

Setelah proses pembuatan tabel selesai, alur selanjutnya adalah memverifikasi apakah data benar-benar sudah tersalin. Saya menggunakan perintah select bintang from employees. Perintah dasar ini meminta database untuk menampilkan seluruh baris dan kolom yang ada di dalam tabel tersebut. Dari hasil yang muncul di layar, saya bisa memastikan bahwa tabel employees saya sudah terisi dengan data pegawai yang lengkap.

```
INSERT INTO employees VALUES (100, 'Steven', 'King', 'SKING', '515.123.4567', TO_DATE('17-JUN-03', 'dd-MON-yy'), 'AD_PRES', 24000, NULL, NULL, 90);
INSERT INTO employees VALUES (101, 'Neena', 'Kochhar', 'NKOCHHAR', '515.123.4568', TO_DATE('21-SEP-05', 'dd-MON-yy'), 'AD_VP', 17000, NULL, 100, 90);
INSERT INTO employees VALUES (102, 'Lex', 'De Haan', 'LDEHAAN', '515.123.4569', TO_DATE('13-JAN-01', 'dd-MON-yy'), 'AD_VP', 17000, NULL, 100, 90);
INSERT INTO employees VALUES (103, 'Alexander', 'Hunold', 'AHUNOLD', '590.423.4567', TO_DATE('03-JAN-06', 'dd-MON-yy'), 'IT_PROG', 9000, NULL, 102, 60);
INSERT INTO employees VALUES (104, 'Bruce', 'Ernst', 'BERNST', '590.423.4568', TO_DATE('21-MAY-07', 'dd-MON-yy'), 'IT_PROG', 6000, NULL, 103, 60);

COMMIT;

PROMPT === ISI TABEL EMPLOYEES ===
SELECT * FROM employees;
```

Script Output | Query Result | All Rows Fetched: 5 in 0.019 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-06-2003	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-09-2005	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-01-2001	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-01-2006	IT_PROG	9000	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21-05-2007	IT_PROG	6000	(null)	103	60

Nomor 3

Langkah ketiga ini cukup kompleks karena saya mulai membuat sebuah program kecil di dalam database yang disebut procedure, dengan nama increase_salary. Procedure ini saya rancang agar bisa menerima dua nilai saat dipanggil, yaitu ID departemen dan seberapa besar persentase kenaikan gajinya.

Cara kerja procedure ini bermula dari proses pencarian data. Program akan melakukan perulangan untuk mengecek satu per satu pegawai yang bekerja di departemen sesuai inputan. Jika ditemukan pegawai yang cocok, program akan menjalankan perintah update untuk menaikkan gajinya. Rumus yang digunakan adalah gaji lama ditambah dengan hasil perkalian antara gaji lama dan persentase kenaikan. Agar saya tahu prosesnya berjalan, saya juga menyisipkan perintah untuk mencetak teks di layar setiap kali ada satu data pegawai yang berhasil diperbarui.

```
CREATE OR REPLACE PROCEDURE increase_salary (
    department_id_in IN employees.department_id%TYPE,
    increase_pct_in IN NUMBER
)
IS
BEGIN
    FOR employee_rec IN (
        SELECT employee_id
        FROM employees
        WHERE department_id = increase_salary.department_id_in
    )
    LOOP
        UPDATE employees emp
        SET emp.salary = emp.salary + (emp.salary * increase_salary.increase_pct_in)
        WHERE emp.employee_id = employee_rec.employee_id;

        DBMS_OUTPUT.PUT_LINE ('Updated ' || SQL%ROWCOUNT);
    END LOOP;
END increase_salary;
/
```

Script Output × | Query Result ×

Task completed in 0.582 seconds

```
Commit complete.

=== ISI TABEL EMPLOYEES ===
>>Query Run In:Query Result

Procedure INCREASE_SALARY compiled
```

Nomor 4

Setelah procedure tersebut berhasil dibuat tanpa error, saya melakukan pengujian dengan memanggilnya. Saya memberikan nilai 90 untuk departemennya dan 0.50 untuk persentasenya. Di layar terminal, muncul pemberitahuan bahwa data berhasil diupdate.

Namun, yang menarik di sini adalah tepat setelah memanggil procedure, saya mengeksekusi perintah rollback. Dalam konsep transaksi database, rollback berfungsi seperti tombol undo untuk membatalkan semua perubahan yang belum disimpan permanen. Akibatnya, saat saya mengecek ulang data pegawai di departemen 90 menggunakan perintah select, nilai gaji mereka ternyata masih sama seperti semula karena proses

kenaikan gajinya telah dibatalkan oleh perintah rollback tersebut.

```
BEGIN
    increase_salary (90, .50);
    ROLLBACK;
END;
/

PROMPT === HASIL SELECT SETELAH ROLLBACK (DEPT 90) ===
SELECT * FROM employees WHERE department_id = 90;
```

Script Output | Query Result | Query Result 1

SQL | All Rows Fetched: 3 in 0.018 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-06-2003	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-09-2005	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-01-2001	AD_VP	17000	(null)	100	90

Nomor 5

Berpikir dari kejadian di atas, pertanyaannya adalah apa yang terjadi jika perintah rollback pada langkah keempat tadi saya hapus?

Pemikiran logisnya, tanpa adanya rollback, proses pembatalan transaksi tidak akan pernah terjadi. Artinya, eksekusi kenaikan gaji sebesar 50 persen untuk pegawai di departemen 90 akan benar-benar diterapkan pada data di memori. Jika setelah itu saya menjalankan perintah select untuk melihat datanya, nominal gaji pegawai di departemen tersebut pasti sudah berubah menjadi lebih tinggi dari sebelumnya. Untuk memastikan perubahan tersebut tersimpan secara fisik dan permanen di dalam database, tindakan yang seharusnya dilakukan adalah mengganti perintah rollback dengan perintah commit.